

Course Code : 2301CS301**Date** : 29-11-2025**Course Name** : Data Structure**Duration** : 150 Minutes**Total Marks** : 70**Instructions:**

1. Attempt all the questions.
2. Figures to the right indicates maximum marks.
3. Make suitable assumptions wherever necessary.

Q.1 (A) Define Data Structure, Storage Structure, and File Structure with suitable examples. Classify Data Structures. **4**

(B) Explain the concept of an Array. Write advantages, limitations, and applications of arrays. **3**

OR

Write an algorithm to insert an element at a specific position in a one-dimensional array.

(C) A stack is initially empty. Perform the following sequence of operations: **7**
 PUSH(10), PUSH(20), PUSH(15), POP(), PUSH(25), PUSH(5), POP(), PUSH(30)
 i. Trace the stack contents after each operation.
 ii. Draw the final configuration of the stack.
 iii. Compute the summation of all remaining elements in the stack.
 iv. Find the maximum element and minimum element in the final stack.
 v. If each element in the final stack represents the marks of a student, calculate the average marks.

OR

Evaluate the postfix expression $9\ 3\ *\ 2\ 5\ +\ -\ 4\ 2\ /\ +$ using a stack.

- i. Show the stack contents step by step.
- ii. Write the final result of the expression.
- iii. Justify why postfix evaluation is more efficient than infix evaluation in terms of stack operations.

Q.2 (A) Define a Queue. Differentiate between a linear queue and a circular queue with neat diagram. **4**

(B) Write an algorithm to insert a node at a specific position in a singly linked list. **3**

OR

Write an algorithm to delete a node with a given key value from a doubly linked list.

(C) Scenario: **7**
 A ticket booking system for a cinema hall works as follows:
 1. Customers arrive and take a ticket in order of arrival.
 2. Occasionally, a customer may cancel their booking, and the system should remove that customer efficiently.
 3. The system also needs to frequently access the first and last customer in the booking list.

4. For VIP customers, the system allows priority booking where they are inserted ahead of regular customers without disturbing the order of others.

Tasks:

- (i) Identify which data structure(s) is best suited to implement the above system efficiently. Consider Queue, Linked List, or any combination.
- (ii) Justify your choice for each operation based on time complexity and operational requirements.
- (iii) Draw a diagram showing how the data structure organizes the customer bookings after the following operations:
 - Customer A, B, C arrive (regular booking)
 - Customer D (VIP) arrives
 - Customer B cancels the booking

OR

Scenario:

A text editor maintains the undo-redo functionality for user actions:

1. Every action performed by the user (like typing a character, deleting text, or formatting) is stored in the order it was performed.
2. The user can undo the last action at any time.
3. After an undo, the user can redo the undone action if they choose.
4. Occasionally, the editor needs to display the last 5 actions performed for review.

Tasks:

- (i) Identify the most suitable data structure(s) to implement the undo-redo functionality efficiently.
- (ii) Justify your choice with exact reasoning, considering time complexity, order of operations, and memory usage.
- (iii) Illustrate with a diagram or stack trace showing the sequence of operations after the following actions:
 - a. Type 'A'
 - b. Type 'B'
 - c. Delete 'A'
 - d. Undo last action
 - e. Type 'C'
 - f. Redo last action
- (iv) If each action is assigned a weight (Type = 1, Delete = 2, Undo = 3, Redo = 3), calculate the total weighted actions currently in the undo stack.

- | | | | |
|-------------------|-------------------|--|-----------------|
| <p>Q.3</p> | <p>(A)</p> | <p>(i) Define a Binary Tree and a General Tree</p> <p>(ii) Draw a general tree with the following structure:</p> <ol style="list-style-type: none"> a. Root: A b. A has three children: B, C, D c. B has two children: E, F d. C has one child: G e. D has two children: H, I <p>(iii) Convert the above general tree to a binary tree.</p> | <p>4</p> |
| <p>(B)</p> | | <p>Represent the following graph using:</p> <ul style="list-style-type: none"> • Adjacency Matrix • Adjacency List <p>Graph:</p> <ul style="list-style-type: none"> • Vertices: A, B, C, D • Edges: A–B, A–C, B–C, B–D, C–D | <p>3</p> |

OR

Represent the graph given by:

- Vertices: P, Q, R, S
- Edges: P–Q, P–R, Q–R, R–S

Draw Adjacency Matrix and Adjacency List, and find the total number of edges.

- (C)** (i) Construct a Binary Search Tree (BST) by inserting the following nodes in order: 50, 30, 70, 20, 40, 60, 80 **7**
- (ii) Draw the resulting BST.
- (iii) Perform the following traversals and write the sequences: Inorder, Preorder, Postorder.

OR

- (i) Draw a Trie to store the following words: cat, car, cart, dog, dot.
- (ii) Count the number of nodes in the Trie.
- (iii) Explain how to search for the word "cart" step by step.

- Q.4 (A)** (i) Define a Hash Table and Hash Function. **4**
- (ii) Draw a hash table of size 10 and insert the following keys using hash function $h(\text{key}) = \text{key} \bmod 10$:
- a. 12, 25, 36, 20, 15
- (iii) Show the resulting table after insertion.

- (B)** (i) Draw a sequential file structure for storing student records with fields: **3**
- a. Roll No, Name, Marks
- (ii) Show how the records are stored sequentially.

OR

Draw an indexed file structure for storing the same student records and show the index table.

- (C)** (i) Consider a hash table of size 10 with hash function $h(\text{key}) = \text{key} \bmod 10$. **7**
- (ii) Insert the following keys using linear probing to resolve collisions:
- a. 12, 22, 32, 42, 52
- (iii) Draw the resulting hash table.
- (iv) Compute the sum of all keys stored at index > 5 .

OR

- (i) Consider a hash table of size 7 with hash function $h(\text{key}) = \text{key} \bmod 7$.
- (ii) Insert the keys: 10, 20, 5, 7, 14 using chaining for collisions.
- (iii) Draw the resulting hash table with linked lists for collisions.
- (iv) Compute the sum of all keys stored at index 3 and 6.

- Q.5 (A)** (i) Define Linear Search and Binary Search. **4**
- (ii) Draw a binary search process to find the key 9 in the sorted array: 1, 3, 5, 7, 9, 11, 13

- (B)** Draw the steps of Bubble Sort to sort the array: 5, 3, 8, 2, 4. Show the array after each pass until it is fully sorted. **3**

OR

Draw the steps of Selection Sort for the same array: 5, 3, 8, 2, 4. Show each pass and the array after swapping.

- (C) Consider the array: 6, 3, 8, 2, 5. Sort it using Insertion Sort. After sorting, **7** compute sum of elements at even positions.

OR

Consider the array: 6, 3, 8, 2, 5. Sort it using Quick Sort. After sorting, compute sum of first three elements.
